Programming with Python



8. File I/O

Introduction

- Data files
  - Can be **created, updated**, and **processed** by Python programs
  - Are used for permanent storage of large amounts of data
    - Storage of data in variables and arrays is only temporary

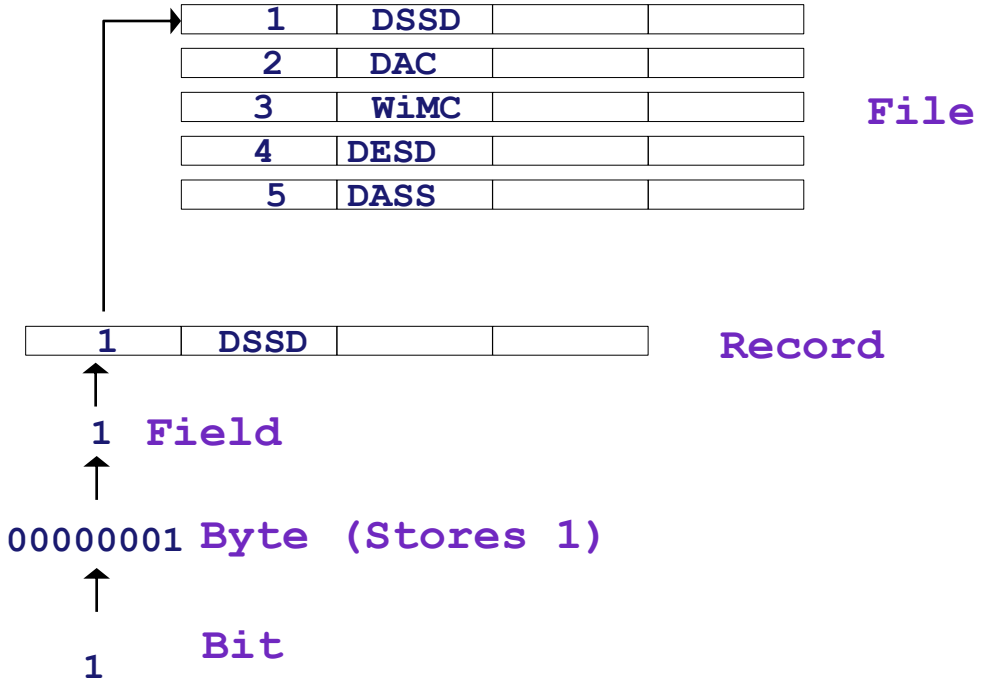
The Data Hierarchy

- Data Hierarchy:
  - Bit – smallest data item
    - Value of 0 or 1
  - Byte – 8 bits
    - Used to store a character
  - Decimal digits, letters, and special symbols
  - Field – group of characters conveying meaning
    - Example: your name
  - Record – group of related fields
    - Represented by a struct or a class

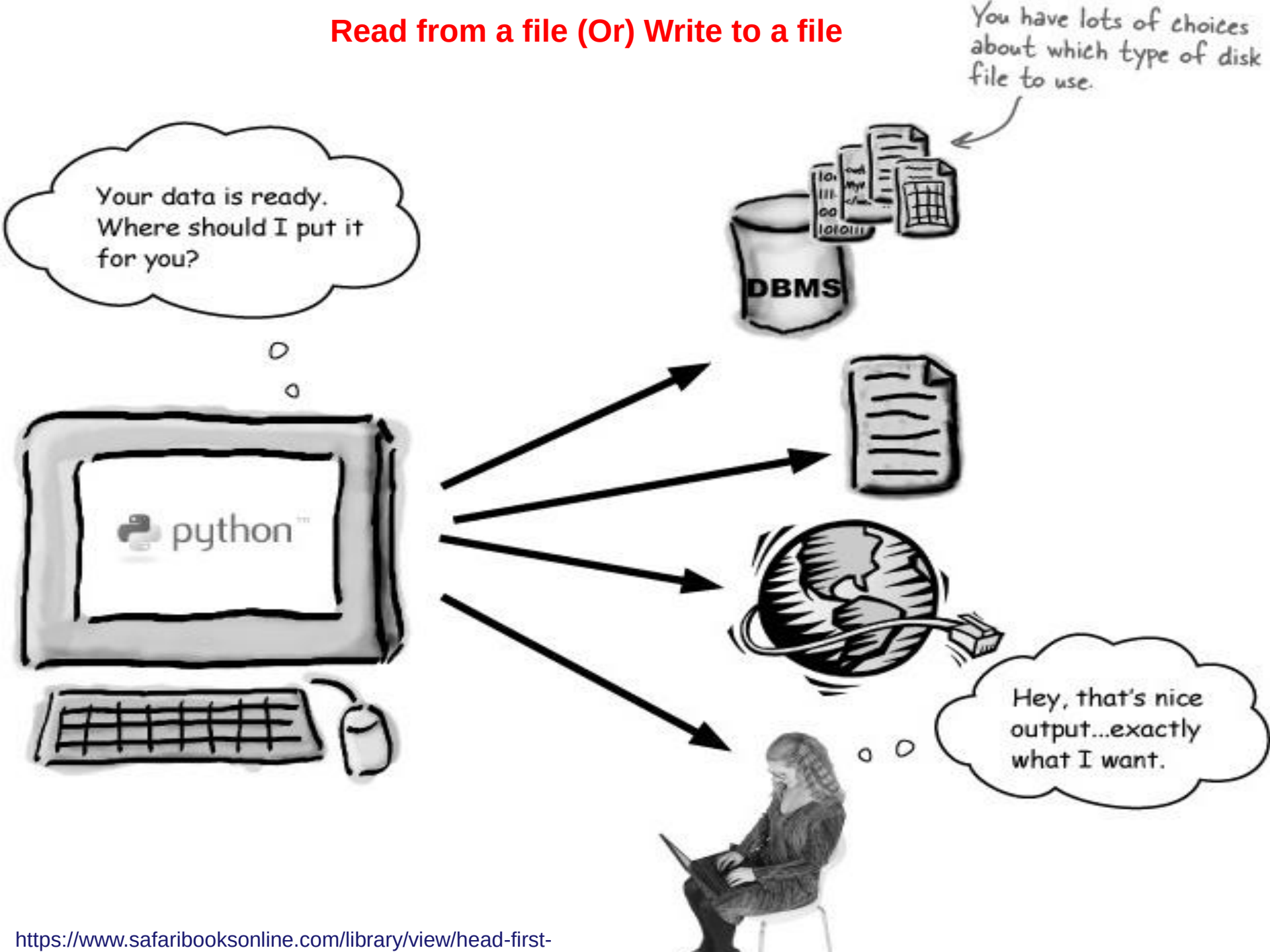
Example: In a payroll system, a record for a particular employee that contained his/her identification number, name, address, etc.

The Data Hierarchy…

- **Data Hierarchy (continued):**
  - **File** – group of related records
    - Example: Employee information file
  - **Database** – group of related files



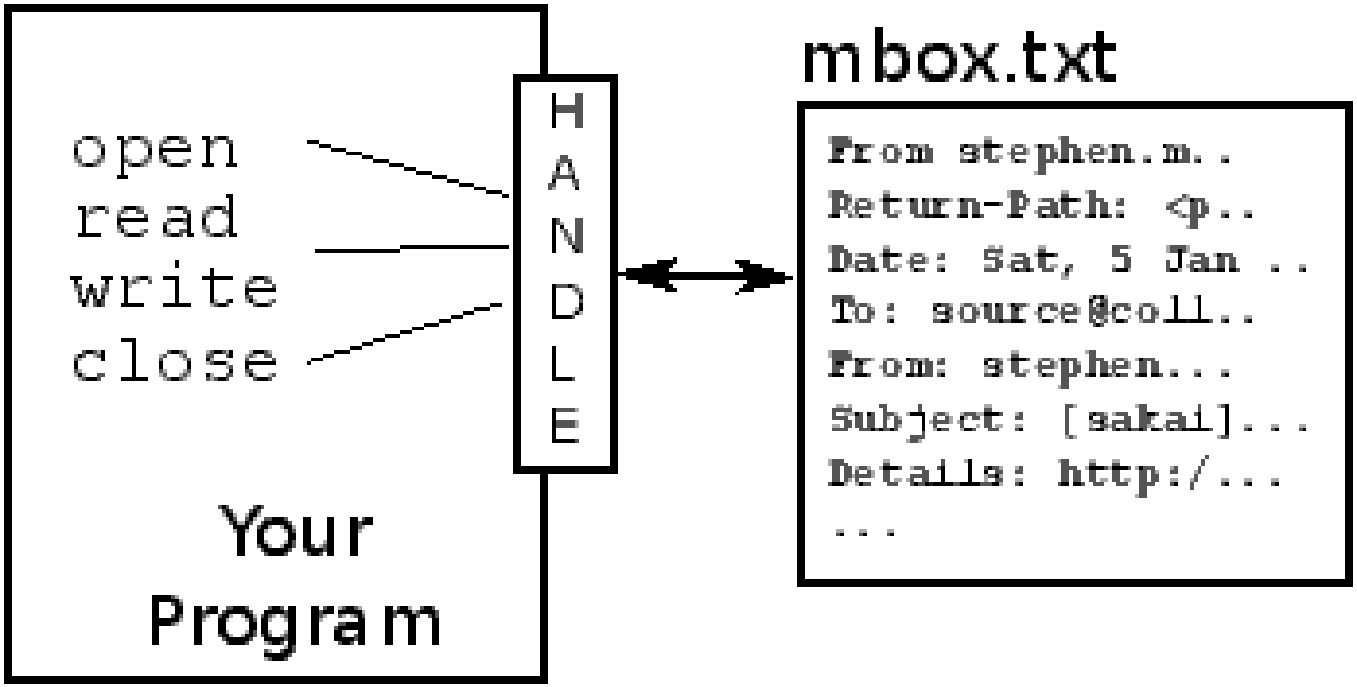
Read from a file (Or) Write to a file



https://www.safaribooksonline.com/library/view/head-first-

Steps

- Open the file
- Read or Write
- Close the file



What tools we have to do that???

List of methods available in Python

Function	Description
open( )	to access files. open() returns a "file handle" - a variable used to perform operations on the file
read( )	read() with no arguments to read the entire file at once
write( )	Write the given size of bytes and returns the number of bytes written
close( )	Closes the opened file
readline(size)	Reads a line from a file, the optional size is the number of bytes to be read from the file.
readlines( )	Reads a line at a time, and returns a list of one-line strings.

File Modes

- '**r**‘ : open for reading (default)
- '**w**' : open for writing, **truncating the file first**
- '**x**' : open for exclusive creation, failing if the file already exists
- 'a' : open for writing, appending to the end of the file if it exists
- '**b**' : binary mode
- '**t**' : text mode (default)
- '+' : open a disk file for updating (reading and writing)

What You Need to do in order to Read from a file??

a. Open the file and associate the file with a file variable.

b. A command to read the information.

c. A command to close the file.

My Data – A Poem

RB_poem.txt

Where the mind is without fear and the head is held high Where knowledge is free Where the world has not been broken up into fragments By narrow domestic walls Where words come out from the depth of truth Where tireless striving stretches its arms towards perfection Where the clear stream of reason has not lost its way Into the dreary desert sand of dead habit Where the mind is led forward by thee Into ever-widening thought and action Into that heaven of freedom, my Father, let my country awake.

a. Open the file

Prepares the file for reading:

A. Links the file variable with the physical file (references to the file variable are references to the physical file).
B. Positions the file pointer at the start of the file.

Format:

<*file variable*> = open(<*file name*>, "r")

Code:

```
inputFile = open(“poem.txt", "r")
```

OR

(Variable file name: entered by user at runtime)

```
filename = input("Enter name of input file: ")
inputFile = open(“filename”, "r")
```

a. Open the file…

Positions the file pointer at the start of the file.

poem.txt

```
Where the mind is
```



```
Without fear and
the head is held high
Where knowledge is
free Where the world
has not been  broken
up into fragments By
narrow domestic walls
Where words come ….
```

b. Read data from the file

- Typically reading is done within the body of a loop
- Each execution of the loop will read a line from the file into a string

Syntax

```
for <var to store a string> in <name of file variable>:
        <Do something with the string read from file>
```

Code:

```
# Echo file contents back onscreen
for line in inputFile:
    print(line)
```

c. Close the file

- Although a file is automatically closed when your program ends it is still a good style to explicitly close your file as soon as the program is done with it.
  - What if the program encounters a runtime error and crashes before it reaches the end? The input file may remain ‘locked’ in an inaccessible state because it’s still open.

Syntax

```
<name of file variable>.close()
```

Code:

```
inputFile.close()
```

Read from a file – All the steps together

```
# Open the file in read mode
inputFile = open("poem.txt", "r")

# Read each line and print to stdout
for line in inputFile:
    print(line)

# Close the file
inputFile.close()
```

2. Write data to a file

Steps:

a. Open the file and associate the file with a file variable (file is “locked” for writing).

b. A command to write the information.

c. A command to close the file.

a) Open the file

Prepare the file for writing.

Format:

<name of file variable> = open(*<file name>*, "w")

Example:

- Constant file name

```
outputFile = open(“names.txt", "w")
```

- Variable file name: entered by user at runtime

```
outputFileName = input("Enter the name of the output
                        file to record the names to: ")
outputFile = open(outputFileName, "w")
```

b. Write data to the file and close

- You can use the ‘write()’ function in conjunction with a file variable.
- **Note:** This function will ONLY take a string parameter (everything else must be converted to this type first).

Syntax

```
Outputfile.write(temp)
```

Code:

```
names  =  ["ram", "John", "Panda"]
for name in names:
    outputFile.write(name)
    outputFile.write("\n")
```

readline()

- The method **readline()**reads one entire line from the file.
- If the *size* argument is present and non-negative, it is a maximum byte count including the trailing newline and an incomplete line may be returned.
- An empty string is returned only when EOF is encountered immediately.

- **Syntax:**
  - fid.readline( )
  - fid.readline(size)

readlines()

- if the file is not too large, it's more convenient to read the file into a complete data structure, e.g. a string or a list.
- **Syntax:**
  - fid.readlines()
- **Example:**

```
emps = open("employees.txt", "r").readlines()
print(emps)
```

Reading files iteratively

- read a text file by using an ***iterator***. This returns one line at a time.
- Similar to readline(), but with less code:

```
poem = ''
fid = open('employees.txt', 'r' )

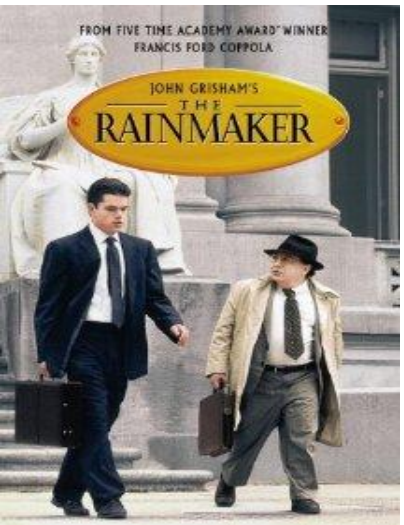
for line in fin:
    print(line)

fid.close()
```

THANK YOU

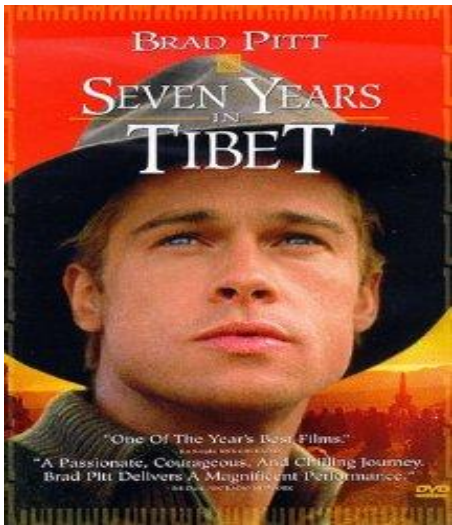
Case Study

Analytics with Movie Ratings









1. OBTAINING DATA SETS AND PREPROCESSING

- **MovieLens** data set collected by **GroupLens** Project of University of Minnesota
- 3 files – movies.dat, ratings.dat, users.dat
- Each user has rated at least 20 movies

Data Set	No. of Ratings	No. of Users	Ratings Range	No .of Movies rated	File Size
1) 100k	100,000 (1 Lakh)	943	1 to 5	1,682	1.8 MB
2) 1M	1,000,209 (~1 Million)	6040	1 to 5	3,900	21 MB
3) 10M	10,000054 (~ 10 Million)	71567	1 to 5	10,681	253 MB

RATINGS.DAT FILE

- All ratings are contained in the file "ratings.dat" and are in the following format:

  UserID::MovieID::Rating::Timestamp

  - UserIDs range between 1 and 6040
  - MovieIDs range between 1 and 3952
  - Ratings are made on a 5-star scale (whole-star ratings only)
  - Timestamp is represented in seconds since the epoch as returned by time(2)
  - Each user has at least 20 ratings

Case Study

- Perform the following operations on a movie dataset.
  - List all the movies which are released in 1995
  - What is number of animation movies?
  - List all the horror movies with starting letter as either D or J
  - What is the number of action movies which are released during 1991-1995?
  - Find out 10 movies with highest disagreement between men and women.
  - What is the favorite genre of programmers?

Thank You